Génération de Texte avec LSTM

Amélioration d'un modèle RNN pour la génération de texte

UCE - Analyse et Modélisation des Systèmes

Master 1 Informatique

Sommaire

1. **Objectifs** du projet
2. **Architecture** : GRU vs LSTM
3. **Améliorations** apportées
4. **Résultats** et démonstration
5. **Difficultés** rencontrées
6. **Conclusion**

Objectifs du Projet

Point de départ

- Script `articleGeneration.py` utilisant un **GRU**
- Génération de texte caractère par caractère
- Entraînement sur le corpus Shakespeare

Mission

- Remplacer GRU par **LSTM**
- Ajouter le support **GPU Apple Silicon (MPS)**
- Implémenter un **split train/validation/test**
- Calculer la **perplexité**

Architecture : GRU vs LSTM

GRU (Gated Recurrent Unit)

- **2 portes** : Reset et Update
- **1 seul état** : hidden state (h)
- Plus simple, plus rapide

LSTM (Long Short-Term Memory)

- **3 portes** : Forget, Input, Output
- **2 états** : hidden state (h) + **cell state (c)**
- Meilleure mémoire long terme

Schéma : GRU vs LSTM

```
┌──────────────────────────────────────────────────────────────┐
│                          GRU                                 │
│                                                              │
│    h(t-1) ──────────────────────────────────────> h(t)       │
│              │                            │                  │
│         [Reset Gate]              [Update Gate]              │
│                                                              │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                          LSTM                                │
│                                                              │
│    c(t-1) ──────────────────────────────────────> c(t)       │  ← Cell State (mémoire long terme)
│              │           │              │                    │
│         [Forget]    [Input]       [Output]                   │
│              │           │              │                    │
│    h(t-1) ──────────────────────────────────────> h(t)       │  ← Hidden State (mémoire court terme)
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Changement dans le Code

```python
# AVANT : GRU
class RNN(nn.Module):
    def __init__(self, ...):
        self.gru = nn.GRU(hidden_size, hidden_size, n_layers)

    def init_hidden(self):
        return torch.zeros(n_layers, 1, hidden_size)  # 1 état
```

```python
# APRÈS : LSTM
class RNN(nn.Module):
    def __init__(self, ...):
        self.lstm = nn.LSTM(hidden_size, hidden_size, n_layers)

    def init_hidden(self):
        h_0 = torch.zeros(n_layers, 1, hidden_size)  # hidden
        c_0 = torch.zeros(n_layers, 1, hidden_size)  # cell
        return (h_0, c_0)  # tuple de 2 états
```

Amélioration 1 : Support GPU

Avant (GRU)

- Uniquement **CUDA** (NVIDIA)

Après (LSTM)

- CUDA + **Apple Silicon (MPS)**

```
if torch.cuda.is_available():
    device = torch.device("cuda:0")       # GPU NVIDIA
elif torch.backends.mps.is_available():
    device = torch.device("mps")          # GPU Apple M1/M2/M3
else:
    device = torch.device("cpu")
```

> **Gain** : Entraînement 10-100x plus rapide sur GPU

Amélioration 2 : Split Train/Val/Test

Avant

- 100% du corpus pour l'entraînement
- Pas de détection d'overfitting

Après

Ensemble	Ratio	Utilisation
Train	80%	Apprentissage du modèle
Validation	10%	Détection overfitting
Test	10%	Évaluation finale

> **Avantage** : Évaluation honnête sur des données jamais vues

Amélioration 3 : Perplexité

Formule

$$\text{Perplexité} = e^{\text{CrossEntropyLoss}}$$

Interprétation

- **Perplexité = 100** : Le modèle est équivalent au hasard
- **Perplexité = 10** : Hésite entre ~10 caractères
- **Perplexité = 2** : Presque certain

Objectif atteint

Perplexité < 15 sur le test set

Amélioration 4 : Sauvegarde Intelligente

Avant

- Sauvegarde **à la fin** de l'entraînement
- Risque de sauvegarder un modèle sur-entraîné

Après

- Sauvegarde quand la **validation loss s'améliore**
- On garde **le meilleur modèle**

```
if val_loss < best_val_loss:
    best_val_loss = val_loss
    torch.save(decoder, modelFile)
    print('Meilleur modèle sauvegardé!')
```

Récapitulatif des Améliorations

Aspect	GRU (Avant)	LSTM (Après)
Architecture	GRU (1 état)	LSTM (2 états)
GPU	CUDA seul	CUDA + MPS
Séquences	13 chars	**100 chars**
Learning rate	0.005	**0.002**
Hidden size	128	**256**
Validation	Non	Oui
Perplexité	Non	Oui
Sauvegarde	Fin	**Meilleur modèle**

Démonstration

Commande d'entraînement

```
python articleGeneration_LSTM.py --trainEval train -d data/shakespeare.txt
```

Commande de génération

```
python articleGeneration_LSTM.py --trainEval eval --length 500
```

Exemple de génération

> **Input** : "The"
> **Output** : "There and this merchio: And my true master. BENVOLIO: Well it a the care..."

Difficultés Rencontrées

1. Détection du GPU Apple Silicon

- MPS est récent dans PyTorch
- Certaines opérations non supportées initialement

2. Hyperparamètres

- Trouver le bon équilibre `hidden_size` / `num_layers`
- Learning rate trop élevé : instabilité

3. Overfitting

- Résolu avec le split train/val/test
- Sauvegarde du meilleur modèle sur validation

Conclusion

Ce que j'ai appris

- Différence **GRU vs LSTM** (portes, états)
- Importance du **split train/val/test**
- Métrique de **perplexité** pour les modèles de langage
- Optimisation **GPU** (CUDA, MPS)

Résultats

- Perplexité < 15 sur le test set
- Génération de texte cohérent style Shakespeare
- Code documenté et modulaire